

## **Ros Day02 hw3 보고서**

작성자: Doyeon Park

작성일: 2026.09.10

## 목차

1. 과제 목적
2. 패키지 구성
3. Qt 구성
4. ROS2 통신 구현
5. Talker 와 Listener
6. 실행 결과

## 1. 과제 목적

Qt GUI 에서 문자열을 입력하고 버튼을 눌러 메시지를 발행하는 기능을 구현하고, ROS2 의 publisher 와 subscriber 를 이용하여 두 노드 사이에서 문자열을 전달하였다.

또한 Listener 가 메시지를 받으면 수신한 문자열을 Qt 라벨에 표시하도록 구현하였다.

Qt 의 signal-slot 방식과 ROS2 토픽 통신을 함께 사용해보는 것이 목적이다.

## 2. 패키지 구성

패키지 이름은 parkdoyeon\_day2\_hw3 이며 ament\_cmake 방식의 C++ 패키지로 생성하였다. 프로그램은 Talker 와 Listener 를 각각 실행할 수 있도록 구성하였으며, 소스 파일과 헤더 파일을 분리하였다.

talker\_window.hpp 에는 TalkerWindow 클래스와 메시지 발행 함수, ROS 2 Publisher, 문자열 입력창, 상태 라벨, Qt Timer 에 필요한 변수들이 선언되어 있다.

talker\_window.cpp 에는 Talker GUI 구성, 버튼 입력 처리, 문자열 메시지 생성 및 발행 기능이 구현되어 있다.

talker\_main.cpp 에서는 ROS 2 와 QApplication 을 초기화하고 qt\_talker 노드와 TalkerWindow 창을 실행한다.

listener\_window.hpp 에는 ListenerWindow 클래스와 메시지 수신 콜백 함수, ROS 2 Subscriber, 수신 메시지 라벨, Qt Timer 에 필요한 변수들이 선언되어 있다.

listener\_window.cpp 에는 Listener GUI 구성, 문자열 메시지 구독, 수신한 문자열을 라벨에 표시하는 기능이 구현되어 있다.

listener\_main.cpp 에서는 ROS 2 와 QApplication 을 초기화하고 qt\_listener 노드와 ListenerWindow 창을 실행한다.

CMakeLists.txt 에는 rclcpp, std\_msgs, Qt5 Widgets 의존성을 추가하였다. Talker 와 Listener 를 각각 별도의 실행 파일로 생성한 뒤 ros2 run 명령으로 실행할 수 있도록 설정하였다. Qt 의 Q\_OBJECT 가 포함된 헤더 파일을 처리하기 위해 CMAKE\_AUTOMOC 도 활성화하였다.

## 3. Qt 구성

Talker GUI 는 제목, 안내 문구, 문자열 입력창, Publish 버튼, 발행 상태 라벨로 구성하였다.

문자열 입력에는 QLineEdit 를 사용하였다. 입력창에는 “메시지를 입력하세요” 라는 안내 문구가 표시되며, 문자열을 입력한 뒤 Publish 버튼을 누르면 메시지가 발행된다.

Publish 버튼은 QPushButton 으로 생성하였다. 버튼의 clicked Signal 을 TalkerWindow 의 publishMessage Slot 에 연결하여 버튼을 누르면 메시지 발행 함수가 실행되도록 하였다.

```
connect(  
publish_button, &QPushButton::clicked,  
this, &TalkerWindow::publishMessage);
```

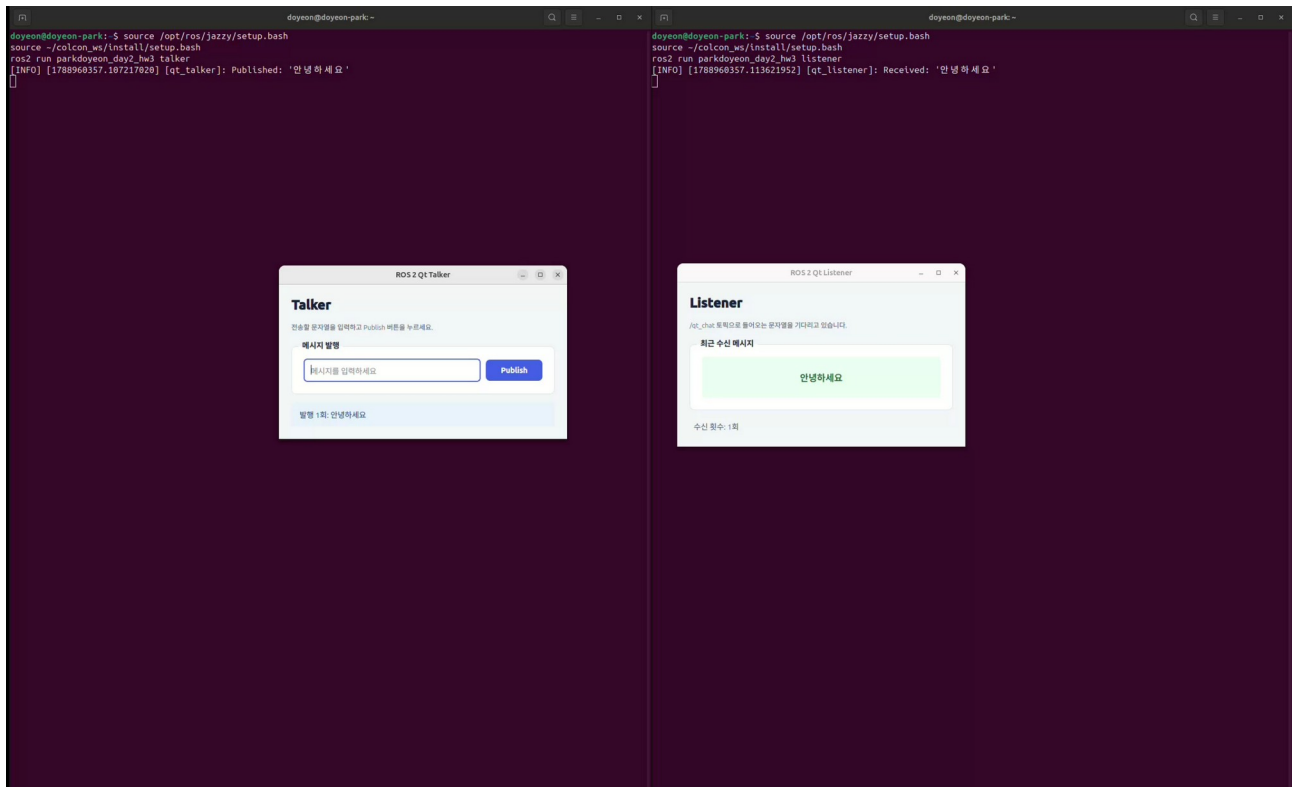
입력창에서 Enter 키를 눌러도 같은 기능을 사용할 수 있도록 QLineEdit 의 returnPressed Signal 도 publishMessage 함수에 연결하였다.

Talker 화면 아래에는 지금까지 발행한 횟수와 가장 최근에 발행한 문자열이 표시된다. 메시지를 발행한 뒤에는 입력창을 비우고 다시 입력창에 포커스를 설정하도록 하였다.

Listener GUI 는 제목, 안내 문구, 최근 수신 메시지 라벨, 수신 횟수 라벨로 구성하였다.

처음 실행했을 때는 “수신 대기 중...”이라는 문구가 표시된다. /qt\_chat Topic 으로 문자열이 들어오면 초록색 라벨의 내용이 수신한 문자열로 변경된다. 메시지를 받을 때마다 수신 횟수도 1 씩 증가하여 화면에 표시된다.

GUI 의 각 위젯은 createUi 함수에서 생성하였다. QVBoxLayout 과 QHBoxLayout 을 사용하여 위젯을 배치하고, StyleSheet 를 적용하여 버튼, 입력창, 라벨의 색상과 여백을 설정하였다.



## 4. ROS2 통신구현

Talker 와 Listener 사이의 문자열 통신에는 /qt\_chat Topic 을 사용하였다. 메시지 자료형은 std\_msgs/msg/String 으로 설정하였다.

Talker 에서는 create\_publisher 함수를 이용하여 Publisher 를 생성하였다.

```
publisher_ = node_->create_publisher<std_msgs::msg::String>(
"/qt_chat", 10);
```

메시지를 발행할 때는 입력창의 문자열을 가져온 뒤 앞뒤의 공백을 제거하였다. 입력값이 비어 있으면 메시지를 발행하지 않고 “문자열을 먼저 입력하세요.”라는 안내를 Talker 화면에 표시하도록 하였다.

입력된 문자열은 Qt 의 QString 자료형이므로 toString 함수를 사용하여 std::string 으로 변환하였다. 이후 std\_msgs::msg::String 메시지의 data 에 문자열을 저장하고 publish 함수를 호출하였다.

```
std_msgs::msg::String message;  
message.data = input_text.toStdString();  
publisher_->publish(message);
```

메시지를 발행한 뒤 Talker 터미널에도 실제로 전송한 문자열이 출력되도록 하였다.

Listener에서는 create\_subscription 함수를 이용하여 /qt\_chat Topic 을 구독하였다. 메시지가 도착하면 messageCallback 함수가 자동으로 실행되도록 연결하였다.

```
subscription_ = node_->create_subscription<std_msgs::msg::String>(  
    "/qt_chat", 10,  
    std::bind(  
        &ListenerWindow::messageCallback,  
        this,  
        std::placeholders::_1));
```

Talker 와 Listener 에서 같은 Topic 이름과 메시지 자료형을 사용해야 정상적으로 통신할 수 있다.

messageCallback 함수에서는 메시지의 data 에 저장된 std::string 을 QString 으로 변환하였다. 변환된 문자열은 setText 함수를 이용하여 Listener 의 최근 수신 메시지 라벨에 표시하였다.

```
const QString received_text =  
    QString::fromStdString(message->data);  
received_message_label_>setText(received_text);
```

메시지를 받을 때마다 Listener 터미널에도 수신한 문자열을 출력하고, receive\_count 값을 증가시켜 수신 횟수를 GUI 에 표시하였다.

## 5. Talker 와 Listener

Qt 는 QApplication 의 Event Loop 를 사용하여 버튼 입력과 화면 갱신을 처리한다. ROS 2 의 Subscriber 는 spin 함수를 실행해야 메시지 수신 콜백을 처리할 수 있다.

일반적인 rclcpp::spin 을 그대로 실행하면 ROS 2 가 실행 흐름을 계속 사용하기 때문에 Qt GUI 가 멈출 수 있다. 이를 방지하기 위해 QTimer 와 rclcpp::spin\_some 을 사용하였다.

QTimer 는 20ms 마다 processRos 함수를 실행하도록 설정하였다.

```
ros_timer_ = new QTimer(this);  
ros_timer_>setInterval(20);  
  
connect(  
    ros_timer_, &QTimer::timeout,  
    this, &TalkerWindow::processRos);  
  
ros_timer_>start();
```

processRos 함수에서는 rclcpp::spin\_some 을 호출하여 현재 도착한 ROS 2 메시지와 콜백만 처리한다.

```
rclcpp::spin_some(node_);
```

이 방식을 사용하면 Qt의 Event Loop가 계속 실행되는 상태에서 ROS 2 메시지도 주기적으로 처리할 수 있다. 따라서 GUI가 멈추지 않으면서 Publish 버튼 입력과 Listener의 라벨 변경 기능이 함께 동작한다.

전체 통신 과정은 다음과 같다.

Talker의 QLineEdit에 문자열 입력

→ Publish 버튼 클릭

→ publishMessage 함수 실행

→ std\_msgs/msg/String 메시지 생성

→ /qt\_chat Topic으로 발행

→ Listener에서 메시지 수신

→ messageCallback 함수 실행

→ 수신한 문자열로 QLabel 변경

## 6. 실행결과

먼저 colcon build 명령을 사용하여 패키지를 빌드하였다.

```
colcon build --packages-select parkdoyeon_day2_hw3
```

빌드 결과 Talker와 Listener 실행 파일이 모두 정상적으로 생성되었다.

두 개의 터미널에서 ROS 2와 작업 공간 환경을 불러왔다.

```
source /opt/ros/jazzy/setup.bash
```

```
source ~/colcon_ws/install/setup.bash
```

첫 번째 터미널에서 Talker를 실행하였다.

```
ros2 run parkdoyeon_day2_hw3 talker
```

두 번째 터미널에서 Listener를 실행하였다.

```
ros2 run parkdoyeon_day2_hw3 listener
```

Talker 입력창에 “안녕하세요”를 입력하고 Publish 버튼을 눌렀다. Talker 터미널에는 Published 로그가 출력되었으며, Listener 터미널에는 같은 문자열의 Received 로그가 출력되었다. Listener의 최근 수신 메시지 라벨도 “안녕하세요”로 변경되었다.

이후 한글 문자열, 숫자, 영어 문자열을 차례대로 입력하여 통신을 확인하였다.

안녕하세요

만나서 반갑습니다

과제 제출

1234

hello

hi

nice to meet you

입력한 모든 문자열이 /qt\_chat Topic 을 통해 Listener 에 정상적으로 전달되었다. Talker 터미널의 Published 내용과 Listener 터미널의 Received 내용이 일치하였으며, Listener GUI 의 라벨도 가장 최근에 수신한 문자열로 변경되었다.

여러 개의 메시지를 연속으로 발행했을 때 Talker의 발행 횟수와 Listener의 수신 횟수가 동일하게 증가하였다. 한글, 숫자, 영어가 포함된 문자열도 모두 정상적으로 전달되는 것을 확인하였다.

이를 통해 Qt의 Signal-Slot을 이용한 버튼 입력, ROS 2 Publisher와 Subscriber를 이용한 문자열 Topic 통신, 수신 메시지를 이용한 Qt 라벨 변경 기능이 모두 정상적으로 동작하는 것을 확인하였다.

The image is a composite of four screenshots arranged in a 2x2 grid, illustrating ROS 2 communication.

**Top Left: Terminal Output for Talker**

```
doyeon@doyeon-park: ~
source /opt/ros/jazzy/setup.bash
source ~/colcon_ws/install/setup.bash
ros2 run parkdoyeon_day2_hw3 talker
[INFO] [1788960357.107217020] [qt_talker]: Published: '안녕하세요.'
[INFO] [1788960372.566391695] [qt_talker]: Published: '안녕하세요.'
[INFO] [1788960376.074899344] [qt_talker]: Published: '안녕하세요입니다.'
[INFO] [1788960378.235782792] [qt_talker]: Published: '귀하 계중.'
[INFO] [1788960381.207870746] [qt_talker]: Published: '1234.'
[INFO] [1788960382.747406769] [qt_talker]: Published: 'hello.'
[INFO] [1788960383.487286842] [qt_talker]: Published: 'hi.'
[INFO] [1788960386.45663352] [qt_talker]: Published: 'nice to meet you.'
```

**Top Right: Terminal Output for Listener**

```
doyeon@doyeon-park: ~
source /opt/ros/jazzy/setup.bash
source ~/colcon_ws/install/setup.bash
ros2 run parkdoyeon_day2_hw3 listener
[INFO] [1788960357.113621952] [qt_listener]: Received: '안녕하세요.'
[INFO] [1788960372.573214555] [qt_listener]: Received: '안녕하세요.'
[INFO] [1788960376.093889463] [qt_listener]: Received: '안녕하세요입니다.'
[INFO] [1788960378.253770434] [qt_listener]: Received: '귀하 계중.'
[INFO] [1788960381.2133091421] [qt_listener]: Received: '1234.'
[INFO] [1788960382.753232382] [qt_listener]: Received: 'hello.'
[INFO] [1788960383.493675834] [qt_listener]: Received: 'hi.'
[INFO] [1788960386.473477872] [qt_listener]: Received: 'nice to meet you.'
```

**Bottom Left: ROS 2 Qt Talker GUI**

The window is titled "ROS 2 Qt Talker". It contains a text input field with the placeholder "메시지를 입력하세요" (Enter message here). To the right of the input field is a blue "Publish" button. Below the input field, it says "발행 8회: nice to meet you" (Published 8 times: nice to meet you).

**Bottom Right: ROS 2 Qt Listener GUI**

The window is titled "ROS 2 Qt Listener". It displays the received message "nice to meet you" in a green box. Above the box, it says "최근 수신 메시지" (Latest received message). Below the box, it says "수신 횟수: 8회" (Received count: 8 times).